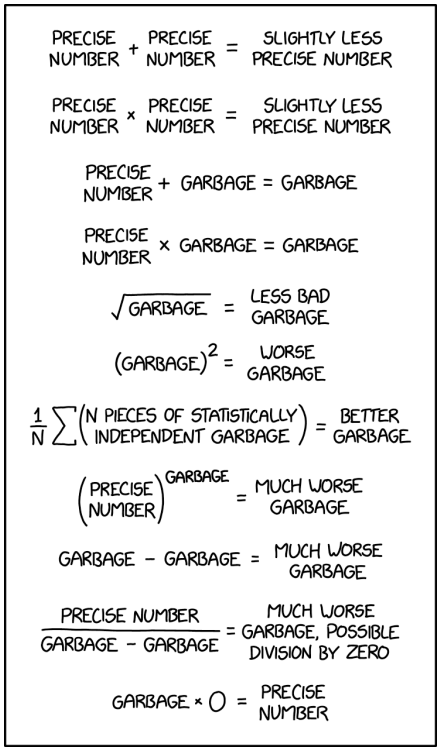
Reference and Practice Handout

Lecture-05: Integer & Floating-Point Arithmetic

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur



Garbage Math

(Randall Munroe; CC BY-NC 2.5; https://xkcd.com/2295/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

This handout will cover:

- Reading **unsigned** and **two's complement** integers, and *sign extension*.
- Converting *fractions* between bases, and reading binary as *hex* and *octal*.
- *Adding* and *subtracting* integers, and spotting **overflow**.
- Storing real numbers with **IEEE 754** floating point (*sign*, *exponent*, *fraction*).
- *Special values*: zero, denormals, infinity, and NaN.
- Floating-point *addition* and *multiplication*, step by step, with **rounding**.
- Why floating point is an *approximation*, and how to compare floats safely.

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

1. Integers (A Quick Refresher)

A row of bits is a number only once you pick a rule. **Unsigned** says every bit adds a positive power of 2. **Two's complement** says the top bit is *negative*. Same bits, different rule, different number.

1.1. Unsigned integers

Each bit x_i is worth 2^i. For an n-bit number:

$$\text{value} = x_{n-1}2^{n-1} + \cdots + x_1 2^1 + x_0 2^0, \qquad \text{range: } 0 \text{ to } 2^n - 1$$

Example 1: Reading 1011_2

$$1011_2 = 1\times 2^3 + 0\times 2^2 + 1\times 2^1 + 1\times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

1.2. Two's complement (signed integers)

The top bit (bit 31 in a 32-bit register) is the **sign bit**: 1 means negative, 0 means non-negative. Non-negative numbers look exactly the same as in unsigned.

Landmark	32-bit pattern	Value
zero	`0000 0000 ... 0000`	0
minus one	`1111 1111 ... 1111`	-1
most negative	`1000 0000 ... 0000`	$-2^{31} = -2{,}147{,}483{,}648$
most positive	`0111 1111 ... 1111`	$2^{31} - 1 = 2{,}147{,}483{,}647$

Shortcut to negate a number: flip every bit, then add 1.
$+6 =$ `0000 0110` $\rightarrow$ flip `1111 1001` $\rightarrow$ $+1$ `1111 1010` $= -6$ (8-bit)

1.3. Sign extension: making a number wider

When a small value moves into a bigger register, the new bits on the left must keep the value the same.

- **Signed:** copy the sign bit into all new positions.
- **Unsigned:** fill new positions with 0s (*zero extension*).

Example 2: 8-bit $\rightarrow$ 16-bit

$+2$: `0000 0010` $\rightarrow$ `0000 0000 0000 0010`
-2: `1111 1110` $\rightarrow$ `1111 1111 1111 1110`

In RISC-V: `lb` sign-extends the loaded byte; `lbu` zero-extends it.

Exercise 1

Memory holds the byte `0xFB`. What 32-bit value ends up in the register after (a) `lb`, and (b) `lbu`? Give each in hex and decimal.

2. Fractions in Other Bases

Digits to the *right* of the point are worth *negative* powers of the base. In binary: $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \ldots$

$$0.47_{10} = 4\times 10^{-1} + 7\times 10^{-2} \qquad 0.11_2 = 1\times 2^{-1} + 1\times 2^{-2} = 0.5 + 0.25 = 0.75_{10}$$
$$11.001_2 = 2 + 1 + 0 + 0 + 0.125 = 3.125_{10}$$

Not every fraction has an exact form in every base. $\frac{1}{2}$ is exact in binary (`0.1`) and decimal (0.5), but not in base 3. Likewise 0.1_{10} has *no* exact binary form: it repeats forever ($0.000110011\ldots_2$). This is the root of many floating-point surprises (Section 8).

2.1. Decimal fraction → binary: the multiply-by-2 method

Multiply by 2. The digit that pops out *in front of* the point is your next bit. Keep the fraction part and repeat until it is 0 (or you have enough bits).

Example 3: Convert 0.8125 to binary

Multiply	Result	Bit out
0.8125×2	1.625	**1**
0.625×2	1.25	**1**
0.25×2	0.5	**0**
0.5×2	1.0	**1**

Read the bits **top to bottom**: $0.8125_{10} = 0.1101_2$.

The same method works for any base: multiply by the target base instead of 2.

Alternative (subtraction method): subtract the largest power $2^{-1}, 2^{-2}, \ldots$ that fits, writing 1 when you subtract and 0 when you skip. For 0.8125: -0.5 (1), -0.25 (1), skip 0.125 (0), -0.0625 (1) $\rightarrow 0.1101_2$.

Exercise 2

Convert 0.375 to binary. Then try 0.1 for six steps. What do you notice?

2.2. Hex and octal: shorthand for binary

Long binary strings are hard to read. Because $16 = 2^4$ and $8 = 2^3$, you can convert by **grouping bits from the right**: groups of 4 for hex, groups of 3 for octal. Pad the left with 0s if needed.

Example 4: $11010100011011_2 = 13595_{10}$

Hex (groups of 4): `0011 0101 0001 1011` → `3 5 1 B` → `0x351B`
Octal (groups of 3): `011 010 100 011 011` → `3 2 4 3 3` → 32433_8

Exercise 3

Write 101101110_2 in hex and in octal.

3. Integer Addition, Subtraction & Overflow

Binary addition works like decimal: add column by column from the right, and carry when a column overflows. In one column, the **sum bit** of $X + Y$ (ignoring the carry) is $X \oplus Y$ (XOR):

X	Y	sum bit $Z = X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0 (carry 1)

Example 5: $7 + 6$ in 8 bits

```
    0000 0111   (+7)
+   0000 0110   (+6)
-------------------
    0000 1101   (+13)
```

Subtraction is addition of the negation: $7 - 6 = 7 + (-6)$.

`0000 0111` + `1111 1010` = `(1) 0000 0001` = $+1$ (the carry out of the top bit is dropped)

3.1. Overflow: when the answer does not fit

Overflow happens when the true result is outside the range the bits can hold. For signed numbers, the tell-tale sign is that the result has the *wrong sign*.

Operation	Overflow if. . .
$(+)+(-)$ or $(-)+(+)$	never
$(+)+(+)$	result sign is 1 (looks negative)
$(-)+(-)$	result sign is 0 (looks positive)
$(+)-(+)$ or $(-)-(-)$	never
$(-)-(+)$	result sign is 0
$(+)-(-)$	result sign is 1

Example 6: $100+50$ in 8-bit signed (range -128 to $+127$)

`0110 0100` + `0011 0010` = `1001 0110`

Two positives gave a result with sign bit 1. The bits read as -106, not 150. **Overflow.**

Hardware check: overflow = (carry *into* the sign bit) $\oplus$ (carry *out of* the sign bit).

carry-in	carry-out	overflow
0	0	0
0	1	1
1	0	1
1	1	0

Watch out

What happens on overflow depends on the machine and the language. C does not check for overflow (unsigned arithmetic wraps around; signed overflow is undefined behavior). Languages such as Ada and Fortran require an exception. MIPS offers both styles (`add`/`addi`/`sub` trap; `addu`/`addiu`/`subu` do not); on a trap the PC is saved in the EPC register and control jumps to a handler. **RISC-V never traps on integer overflow.** Software must check explicitly, e.g., with a branch or `slt`.

Exercise 4

In 8-bit signed arithmetic, which of these overflow? (a) $-100+(-50)$ (b) $7+(-6)$ (c) $-128+1$

4. Floating Point: Scientific Notation in Binary

Integers can't hold 0.75 or 6.626×10^{-34}. Floating point stores a number the way scientists write it: a **sign**, a **significand**, and an **exponent**, but in base 2.

$$\underbrace{+}_{\text{sign}}\ \underbrace{6.626}_{\text{significand}}\times\underbrace{10}_{\text{base}}\overbrace{{}^{-34}}^{\text{exponent}} \quad\longrightarrow\quad (-1)^S\times 1.M\times 2^{E-\text{bias}}$$

Color key used below:

sign S **exponent E** (biased) **fraction M** (a.k.a. mantissa)

4.1. IEEE 754 single precision (32 bits)

S	exponent E	fraction M
bit 31 (1 bit)	bits 30–23 (8 bits)	bits 22–0 (23 bits)

$$\text{value} = (-1)^S \times 1.M \times 2^{E-127}$$

- **Normalized:** the significand is always 1.something, so the leading 1 is *not stored* (the "hidden bit"). You get 24 bits of precision for the price of 23.
- **Biased exponent:** store E = real exponent + 127, so E is always a non-negative number. This makes comparing floats almost as easy as comparing integers.
- The split between E and M is a trade-off: more exponent bits give more *range*; more fraction bits give more *precision*.

Format	Total bits	Exponent bits	Fraction bits	Bias
half	16	5	10	15
single	32	8	23	127
double	64	11	52	1023
quad	128	15	112	16383

4.2. Recipe: decimal → IEEE 754

1. Write the sign: $S = 0$ for positive, 1 for negative.
2. Convert the magnitude to binary (Section 2).
3. Normalize: move the point until the form is $1.\text{xxx} \times 2^k$.
4. Exponent field: $E = k + 127$ (single) or $k + 1023$ (double), written in binary.
5. Fraction field: the bits *after* the point, padded with 0s on the right.

Example 7: Encode -0.75

Steps 1–3: $S = 1$. $0.75 = 0.11_2 = 1.1_2 \times 2^{-1}$, so $k = -1$.
Step 4: single: $E = -1 + 127 = 126 =$ `01111110`; double: $E = -1 + 1023 = 1022 =$ `01111111110`.
Step 5: fraction = `1000...0`.

1	01111110	10000000000000000000000
S	E = 126	M

Single: `0xBF400000`. Double bit pattern starts `1 01111111110 1000...`.

Example 8: Encode 124.67 (not exact!)

$124 =$ `1111100`$_2$ and $0.67 \approx$ `0.1010101110000101...`$_2$ (it never terminates).
$124.67 \approx$ `1111100.10101011...`$_2 =$ `1.11110010101011...`$_2 \times 2^6$, so $S = 0$, $E = 6 + 127 = 133 =$ `10000101`.

0	10000101	11110010101011100001010
S	E = 133	M (rounded to 23 bits)

Single: `0x42F9570A`. The stored value is close to, but not exactly, 124.67.

Exercise 5

Encode 13.25 in single precision. Give the three fields and the hex word.

4.3. Decoding: IEEE 754 → decimal

Split the 32 bits into S, E, M, then compute $(-1)^S \times (1 + 0.M) \times 2^{E-127}$.

Example 9: What is `1 10000001 01000000000000000000000`?

$S = 1$ (negative). $E =$ `10000001` $= 129$, so the real exponent is $129 - 127 = 2$. $M =$ `01` → significand $1.01_2 = 1.25$.

$$x = (-1)^1 \times 1.25 \times 2^2 = -5.0 \qquad (\texttt{0xC0A00000})$$

Exercise 6

Decode `0x3F500000`. (Hint: you met this value in Section 2.)

5. Range, Special Values & Denormals

Two exponent patterns are **reserved**: all 0s and all 1s. Every other exponent ($1 \le E \le 254$ in single) is a normal number.

Exponent E (single)	$M = 0$	$M \neq 0$
$E = 0$	± 0	denormal: $(-1)^S \times 0.M \times 2^{-126}$
$1 \le E \le 254$	$\pm 1.0 \times 2^{E-127}$	normal: $(-1)^S \times 1.M \times 2^{E-127}$
$E = 255$	$\pm\infty$	NaN (Not a Number)

5.1. How big and how small?

	Single (bias 127)	**Double (bias 1023)**
Smallest normal	$E = 1$: $1.0 \times 2^{-126} \approx 1.2 \times 10^{-38}$	$E = 1$: $1.0 \times 2^{-1022} \approx 2.2 \times 10^{-308}$
Largest normal	$E = 254$: $\approx 2.0 \times 2^{127} \approx 3.4 \times 10^{38}$	$E = 2046$: $\approx 2.0 \times 2^{1023} \approx 1.8 \times 10^{308}$

- **Overflow:** the exponent is too large to fit, e.g., 10^{39} in single. The result becomes $\pm\infty$.
- **Underflow:** the exponent is too negative, e.g., 10^{-50} in single. The result is rounded to a denormal or to 0.
- **Gaps:** even inside the range, not every number exists. With 23 fraction bits, $1 + 2^{-24}$ cannot be stored; it rounds to 1.

5.2. Denormal (subnormal) numbers

Without denormals, there would be a sudden jump from 1.2×10^{-38} straight down to 0. Denormals fill that gap so values shrink *gradually* ("gradual underflow").

- Exponent field is all 0s, and the hidden bit becomes **0** instead of 1: value $= 0.M \times 2^{-126}$.
- Largest denormal: M all 1s $\rightarrow 0.111\ldots1_2 \times 2^{-126} = (1 - 2^{-23}) \times 2^{-126}$.
- Smallest denormal: $M =$ `000...01` $\rightarrow 2^{-149} \approx 1.4 \times 10^{-45}$.
- $M = 0$ gives zero, and because S can be 0 or 1, there are **two zeros**: $+0.0$ and -0.0.

5.3. Infinity and NaN

- $\pm\infty$ (E all 1s, $M = 0$): the result of overflow or, e.g., 1.0/0.0. It can flow through later calculations, so code doesn't have to check for overflow after every step.
- **NaN** (E all 1s, $M \neq 0$): an undefined result such as 0.0/0.0 or $\sqrt{-1}$. Any arithmetic with NaN produces NaN.

Exercise 7

Classify each single-precision word as normal, denormal, zero, infinity, or NaN:
(a) `0x7F800000` (b) `0x80000000` (c) `0x00000001` (d) `0xFFC00000`

6. Floating-Point Addition (and Subtraction)

You can only add significands when the exponents match. So: **line up the points**, add, then tidy up (normalize and round).

Example 10: Decimal warm-up with 4 significant digits: $9.999 \times 10^1 + 1.610 \times 10^{-1}$

1. **Align:** shift the number with the smaller exponent right: $1.610 \times 10^{-1} = 0.016 \times 10^1$ (only 4 digits fit).
2. **Add significands:** $9.999 + 0.016 = 10.015$, so 10.015×10^1.
3. **Normalize** (and check overflow/underflow): 1.0015×10^2.
4. **Round** to 4 digits (renormalize if needed): 1.002×10^2.

6.1. The binary algorithm

For $(F_1 \times 2^{E_1}) + (F_2 \times 2^{E_2})$ with $E_1 \geq E_2$:

Step 0: Restore the hidden 1 in F_1 and F_2.
Step 1: Align: shift F_2 right by $E_1 - E_2$ places. Keep the shifted-out bits as G, R, S (Section 6.2).
Step 2: Add F_2 to F_1 (respecting signs) to get F_3.
Step 3: Normalize F_3 to 1.xxx. Same signs: may need one right shift (E_3++, check overflow). Different signs: may need many left shifts (E_3−−, check underflow).
Step 4: Round F_3; if rounding produced 10.xxx, normalize again.
Step 5: Hide the leading 1 again and store.

Example 11: $2.0 + (-0.0546875)$ **with unlimited bits**

$2.0 = 1.0000 \times 2^1$ and $-0.0546875 = -1.1100 \times 2^{-5}$.

- **Align:** shift 1.1100 right by $1 - (-5) = 6$ places: -0.00000111×2^1.
- **Add:** $1.00000000 - 0.00000111 = 0.11111001$, exponent 1.
- **Normalize:** $0.11111001 \times 2^1 = 1.1111001 \times 2^0$. No overflow or underflow.
- **Round:** nothing to round. Result $= 1.1111001_2 = 1.9453125$.

6.2. Rounding with Guard, Round, and Sticky bits

Real hardware keeps only so many bits. When it shifts bits off the right end during alignment, it remembers three of them so it can still round correctly:

- **G (guard):** the first bit past the stored precision.
- **R (round):** the next bit.
- **S (sticky):** 1 if *any* bit after R was 1 (it "sticks" at 1).

Round to nearest (even), using G R S:
- G = 0: round **down** (just drop the extra bits).
- G = 1 and (R or S) = 1: round **up** (add 1 to the last kept bit).
- G = 1, R = 0, S = 0: exact tie. Round so the last kept bit is 0 ("to even").

Example 12: The same sum, $2.0 + (-0.0546875)$**, keeping only 4 fraction bits**

- **Align:** $-1.1100 \times 2^{-5} \to -0.0000\,|\,\underbrace{0}_{G}\,\underbrace{1}_{R}\,\underbrace{1}_{S} \times 2^1$.
  The kept part is 0.0000, and G R S $= 0\,1\,1$ remember what fell off.
- **Add:** $1.0000 - 0.0000011 = 0.1111101$, exponent 1.
- **Normalize:** 1.111101×2^0.
- **Round to 4 bits:** $1.1111\,|\,01$: the bit right after the kept bits is 0, so round down $\to 1.1111 \times 2^0 = 1.9375$.

The exact answer was 1.9453125; the 4-bit result is the closest value that fits.

Exercise 8

Using 4 fraction bits, compute $0.5 + (-0.4375)$ step by step. Write both numbers as $1.\text{xxxx} \times 2^k$ first.

7. Floating-Point Multiplication

Multiplying is actually *simpler* than adding: no alignment. **Add the exponents, multiply the significands, fix the sign.**

Example 13: Decimal warm-up: $(1.110 \times 10^{10}) \times (9.200 \times 10^{-5})$

1. **Add exponents:** $10 + (-5) = 5$.
2. **Multiply significands:** $1.110 \times 9.200 = 10.212$, so 10.212×10^5.
3. **Normalize:** 1.0212×10^6 (check overflow/underflow).
4. **Round** to 4 digits: 1.021×10^6.
5. **Sign:** $+ \times + = +$, so $+1.021 \times 10^6$.

7.1. The binary algorithm

Step 0: Restore the hidden 1 in both significands.
Step 1: Add the *biased* exponents and subtract one bias: $E_3 = E_1 + E_2 - 127$. (Both already contain +127, so one must be removed.) Sign $= S_1 \oplus S_2$.
Step 2: Multiply the significands.
Step 3: Normalize. Both inputs are in $[1, 2)$, so the product is in $[1, 4)$: at most one right shift (E_3++). Check overflow/underflow.
Step 4: Round, and renormalize if needed.
Step 5: Hide the leading 1 and store.

Example 14: $0.90625 \times (-0.28125)$**, keeping 4 fraction bits**

$0.90625 = 1.1101 \times 2^{-1}$ and $-0.28125 = -1.0010 \times 2^{-2}$.

- **Exponent:** real: $-1 + (-2) = -3$. Biased: $126 + 125 - 127 = 124 = -3 + 127$, as expected.
- **Sign:** $+ \times - = -$.
- **Multiply:** $1.1101 \times 1.0010 = 10.0000101$.
- **Normalize:** $10.0000101 \times 2^{-3} = 1.00000101 \times 2^{-2}$ (biased exponent 125).
- **Round to 4 bits:** $1.0000 \,|\, 0101 \rightarrow$ round down $\rightarrow 1.0000$.
- **Result:** $-1.0000 \times 2^{-2} = -0.25$. (Exact product: -0.2548828125.)

Exercise 9

Using 4 fraction bits, compute $0.5 \times (-0.4375)$. Give the biased exponent of the result too.

8. Floating Point Is an Approximation

There are infinitely many real numbers, but only 2^{32} bit patterns. Floating point keeps the *closest* representable value. More bits shrink the error, but never remove it, and errors can **add up** over many operations.

8.1. Ordinary math rules can break

You cannot assume $(a + b) + c = a + (b + c)$, or $a(b + c) = ab + ac$.

Example 15: Order matters (single precision)

Let $a = 10^8$, $b = -10^8$, $c = 1$.

$$(a + b) + c = 0 + 1 = 1 \qquad \text{but} \qquad a + (b + c) = 10^8 + (-10^8) = 0$$

Why? $-10^8 + 1$ rounds back to -10^8: near 10^8, neighbouring floats are 8 apart, so a +1 is lost.

Example 16: Adding many small numbers to a big one (single precision)

Add 0.000005 to 1000000.5, one hundred thousand times. The true answer is 1000001.0.

- **Big first:** each +0.000005 is far smaller than the gap between floats near 10^6 (that gap is 0.0625), so every addition is lost. Result: **1000000.5**.
- **Small first:** add the 100,000 small values to each other (≈ 0.5), *then* add 1000000.5. Result: **1000001.0**.

Lesson: errors are smaller when the operands have *similar magnitudes*.

8.2. Never test floats with ==

Because of rounding, a computed value may be a tiny bit off. In double precision, `0.1 + 0.2 == 0.3` is **false**. Compare with a tolerance ε instead:

```
if (x == 2.0) { ... }              // fragile
if (fabs(x - 2.0) < epsilon) { ... } // better: "close enough"
```

Choosing ε is part of the job: it should match the size of the numbers and how much error the calculation can build up.

Exercise 10

`0.1f` is stored in single precision as `0x3DCCCCCD`. Is the stored value slightly above or below 0.1? Use the fact that 0.1 in binary is $0.0\overline{0011}_2$.

Recap

Remember these

1. Unsigned: every bit is $+2^i$. Two's complement: the top bit is -2^{n-1}. Negate = flip bits, add 1.
2. Sign extension copies the sign bit (`lb`); zero extension fills 0s (`lbu`).
3. Fraction → binary: multiply by 2 and read the integer bits top to bottom. Hex = groups of 4 bits, octal = groups of 3.
4. Signed overflow = the result has the wrong sign = carry-in $\oplus$ carry-out of the sign bit. RISC-V does not trap on it.
5. IEEE 754 single: 1 sign, 8 exponent (bias 127), 23 fraction bits. Value = $(-1)^S \times 1.M \times 2^{E-127}$.
6. $E = 0$: zero or denormal (hidden bit 0). $E = 255$: ∞ or NaN.
7. FP add: align → add → normalize → round. FP multiply: add exponents (minus one bias) → multiply → normalize → round → sign.
8. G, R, S bits let hardware round correctly after shifting.
9. Floating point is approximate: order of operations matters, and equality tests need an ε.

Answer key for Exercises

Ex. 1. (a) `lb`: sign bit of `0xFB` = `1111 1011` is 1 → `0xFFFFFFFB` = -5.
(b) `lbu`: zero-extend → `0x000000FB` = 251.

Ex. 2. $0.375 \to 0.75\,(0),\ 1.5\,(1),\ 1.0\,(1) \to 0.011_2$.
$0.1 \to 0.2\,(0),\ 0.4\,(0),\ 0.8\,(0),\ 1.6\,(1),\ 1.2\,(1),\ 0.4\,(0)\ldots \to 0.000110\ldots_2$. The fraction 0.4 came back, so the pattern `0011` repeats forever: 0.1 has no exact binary form.

Ex. 3. 101101110_2: hex `0001 0110 1110` → `0x16E`; octal `101 101 110` → 556_8. (Both equal 366.)

Ex. 4. (a) `1001 1100` + `1100 1110` = `0110 1010` = +106: two negatives gave a positive → **overflow**.
(b) $7 + (-6) = 1$: mixed signs → **no overflow**.
(c) `1000 0000` + `0000 0001` = `1000 0001` = -127: mixed signs → **no overflow**.

Ex. 5. $13.25 = 1101.01_2 = 1.10101_2 \times 2^3$. $S = 0$, $E = 3 + 127 = 130 =$ `10000010`, $M =$ `10101000...0`.
Word: `0 10000010 10101000000000000000000` = `0x41540000`.

Ex. 6. `0x3F500000` = `0 01111110 10100000000000000000000`: $S = 0$, $E = 126 \to 2^{-1}$, significand $1.101_2 = 1.625$.
$x = 1.625 \times 0.5 = 0.8125$.

Ex. 7. (a) E all 1s, $M = 0$, $S = 0 \to +\infty$. (b) $S = 1$, everything else $0 \to -0.0$.
(c) $E = 0$, $M \neq 0 \to$ smallest denormal, $2^{-149} \approx 1.4 \times 10^{-45}$. (d) E all 1s, $M \neq 0 \to$ NaN.

Ex. 8. $0.5 = 1.0000 \times 2^{-1}$, $-0.4375 = -1.1100 \times 2^{-2}$.
Align: $-1.1100 \times 2^{-2} = -0.1110 \times 2^{-1}$. Add: $1.0000 - 0.1110 = 0.0010$, exponent -1.
Normalize: $0.0010 \times 2^{-1} = 1.0000 \times 2^{-4}$ (three left shifts; $-4 \geq -126$, no underflow). Round: nothing to round.
Result: $1.0000 \times 2^{-4} = 0.0625$.

Ex. 9. Exponents: $-1 + (-2) = -3$; biased $126 + 125 - 127 = 124$. Sign: negative.
Multiply: $1.0000 \times 1.1100 = 1.1100$ (already normalized, nothing to round).
Result: $-1.1100 \times 2^{-3} = -0.21875$, biased exponent 124 = `01111100`.

Ex. 10. $0.1 = 1.\overline{1001}_2 \times 2^{-4}$. The 23 stored fraction bits are `1001 1001 ... 1001 100`, and the next bits are `11...`, so hardware rounds **up** (the last hex digit becomes `D` instead of `C`). The stored value is slightly **above** 0.1: $\approx 0.100000001490116$.

Student Practice Worksheet

Part 1: Integers

Q1. For an 8-bit number, give the range of values as
(a) unsigned: _____ (b) two's complement: _____

Q2. The byte `0x9C` is loaded into a 32-bit register.

- After `lb`: `0x`_____ (= _____ in decimal)
- After `lbu`: `0x`_____ (= _____ in decimal)

Q3. Add the 8-bit signed values `0x70` and `0x50`. Write the 8-bit result in hex: _____. Did it overflow? Explain using the sign rule.

Part 2: Fractions & IEEE 754

Q4. Convert 0.6875 to binary using the multiply-by-2 method. Show each step.

Q5. Encode -0.1875 in IEEE 754 single precision.

`S` (1)	`E` (8)	`M` (23)

Hexadecimal representation: `0x`_____

Q6. What decimal value does `0x41200000` represent? _____

Q7. Explain in one or two sentences why denormal numbers exist.

Part 3: Floating-Point Arithmetic

Q8. Using the 6-step addition algorithm, compute $1.5 + 0.375$. Show the aligned significands.

Q9. Using the multiplication algorithm, compute $1.5 \times (-2.5)$. Give the result's sign, biased exponent, and fraction.

Q10. A program sums a list of floats. Some values are around 10^6 and many are around 10^{-5}. In what order should it add them for the most accurate result, and why?

Step-by-Step Solutions & Explanations

Q1: 8-bit ranges

- Unsigned: 0 to $2^8 - 1 = 255$.
- Two's complement: $-2^7 = -128$ to $2^7 - 1 = +127$.

Q2: Loading `0x9C`

- `0x9C` = `1001 1100`: the top bit is 1.
- `lb`: copy the 1 into the upper 24 bits → `0xFFFFFF9C` = -100.
- `lbu`: fill with 0s → `0x0000009C` = 156.

Q3: `0x70` + `0x50`

- `0x70` = 112, `0x50` = 80; true sum = 192, which is above +127.
- `0111 0000` + `0101 0000` = `1100 0000` = `0xC0`, which reads as -64.
- Two positive operands produced a result with sign bit 1 → **overflow**.

Q4: 0.6875 **to binary**

- $0.6875 \times 2 = 1.375 \to 1$; $0.375 \times 2 = 0.75 \to 0$; $0.75 \times 2 = 1.5 \to 1$; $0.5 \times 2 = 1.0 \to 1$.
- Top to bottom: $0.6875_{10} = \mathbf{0.1011_2}$. Check: $0.5 + 0.125 + 0.0625 = 0.6875$.

Q5: Encode -0.1875

- $S = 1$. $0.1875 = 0.0011_2 = 1.1_2 \times 2^{-3}$.
- $E = -3 + 127 = 124 =$ `01111100`. $M =$ `1000...0`.
- Word: `1 01111100 10000000000000000000000` = `1011 1110 0100 0000 ...` $\implies$ `0xBE400000`.

Q6: Decode `0x41200000`

- `0100 0001 0010 0000 ...` → $S = 0$, $E =$ `10000010` $= 130 \to 2^3$, $M =$ `010...` $\to 1.01_2 = 1.25$.
- $x = 1.25 \times 8 = \mathbf{10.0}$.

Q7: Why denormals? Answer: They fill the gap between 0 and the smallest normal number (2^{-126} in single). Without them, a result just below 2^{-126} would jump straight to 0; with them, values lose precision gradually ("gradual underflow").

Q8: $1.5 + 0.375$

- $1.5 = 1.100 \times 2^0$; $0.375 = 1.100 \times 2^{-2}$.
- Align: shift the smaller right by 2: 0.011×2^0.
- Add: $1.100 + 0.011 = 1.111$, exponent 0. Already normalized; nothing to round.
- Result: $1.111_2 \times 2^0 = \mathbf{1.875}$ (`0x3FF00000`).

Q9: $1.5 \times (-2.5)$

- $1.5 = 1.1 \times 2^0$; $-2.5 = -1.01 \times 2^1$.
- Exponent: $0 + 1 = 1$; biased: $127 + 128 - 127 = \mathbf{128}$. Sign: **negative**.
- Significands: $1.1 \times 1.01 = 1.111$ (already normalized).
- Result: $-1.111_2 \times 2^1 = \mathbf{-3.75}$; fraction field `1110...0` (`0xC0700000`).

Q10: Summation order Answer: Add the small values together first, then add the large values. When a tiny number is added to a huge one, it can be smaller than the gap between neighbouring floats and vanish completely. Summing values of similar magnitude first keeps their contribution.